

Welcome to Course 2: Advanced Learning Algorithms

This course covers two of the most powerful and widely used machine learning algorithms: **Neural Networks** (also called Deep Learning) and **Decision Trees**.

Course Goals

1. **Learn Advanced Algorithms:** Implement and understand neural networks and decision trees.
 2. **Gain Practical Advice:** Learn how to make effective, systematic decisions when building real-world machine learning systems (e.g., deciding between collecting more data vs. building a bigger model) to avoid common pitfalls and wasted time.
-

Course Structure (4 Weeks)

- **Week 1: Neural Networks (Inference)**
 - Understand the fundamentals of how neural networks work.
 - Learn how to use a pre-trained neural network to make predictions (a process called **inference**).
- **Week 2: Neural Networks (Training)**
 - Learn how to train your own neural network from scratch using a labeled dataset (X , Y).
- **Week 3: Practical Advice for Building ML Systems**
 - Focus on the best practices and strategic tips for building efficient and effective ML applications.
- **Week 4: Decision Trees**
 - Learn another highly powerful and widely used algorithm for classification and regression tasks.

Neural Networks Intuition

The Biological Motivation

- **Initial Goal:** Neural networks were originally invented in the 1950s with the goal of mimicking the human brain.
- **The Biological Neuron:** A biological neuron receives input signals via **dendrites**, processes them in its cell body (nucleus), and sends an output signal (an electrical impulse) down an **axon**.¹
- **Modern Context:** This analogy is now considered very loose. Our understanding of the brain is still limited, and modern neural networks are built using engineering principles, not by directly copying biology.

The Artificial Neuron

- An **artificial neuron** is a highly simplified mathematical model of its biological counterpart.²
 - **Function:**
 1. It receives one or more numerical **inputs**.
 2. It performs a simple **computation**.
 3. It produces a single numerical **output**.
 - A **neural network** is created by connecting many of these simple neurons, where the outputs from one layer of neurons become the inputs for the next layer.³
-

A Brief History of Neural Networks

Neural networks have gone through several cycles of popularity and decline (sometimes called "AI winters").⁴

1. **1950s:** Initial invention.
2. **1980s–1990s:** A resurgence in popularity, showing success in applications like handwritten digit recognition (e.g., reading postal codes).⁵
3. **Late 1990s:** Fell out of favor again.
4. **~2005–Present:** A massive resurgence, rebranded as **Deep Learning**.
 - This new wave first revolutionized **speech recognition**.⁶
 - It then had a major impact on **computer vision** (e.g., the 2012 ImageNet competition).⁷
 - It subsequently transformed **Natural Language Processing (NLP)** and many other fields.⁸

Why Are Neural Networks Successful Now? 🚀

Two primary factors are responsible for the recent "deep learning revolution":

1. Massive Data Availability ("Big Data")

- Our digital society (internet, mobile phones) generates enormous amounts of data.⁹
- **Traditional algorithms** (like linear or logistic regression) **plateau** in performance. Their accuracy stops improving even when fed more data.
- **Neural networks scale with data.** Larger neural networks can effectively learn from and leverage massive datasets to achieve much higher performance.¹⁰

2. Faster Computation (Hardware)

- Training large neural networks is computationally very intensive.
- The rise of powerful **GPUs (Graphics Processing Units)**, originally built for video games, provided the perfect hardware for the parallel computations required by neural networks.¹¹

Neural Networks: From Logistic Regression to a Full Network

A Single Neuron as Logistic Regression 🧠

A single **artificial neuron** can be thought of as a simple logistic regression unit.

- **Job:** It takes one or more inputs, performs a computation, and produces a single output.
 - **Example (Demand Prediction):**
 - **Input (x):** Price of a T-shirt.
 - **Output (a):** Probability of it being a "top seller."
 - **Computation:** The standard logistic (sigmoid) function.
 - **New Terminology:** The output of a neuron, $f(x)$, is now referred to as its **activation**, denoted by the letter **a**. This term is borrowed from neuroscience.
-

Building a Neural Network

A neural network is formed by wiring many of these simple neurons together.

- **Example (Complex Demand Prediction):**
 - **Inputs ($\vec{x}$):** A vector of 4 features:
 1. Price (x_1)
 2. Shipping cost (x_2)
 3. Marketing (x_3)
 4. Material quality (x_4)
 - **Goal:** Predict if it's a "top seller" (y).

Layer-by-Layer Computation

1. **Input Layer:** This is not a computational layer; it's just the layer that holds the input feature vector $\vec{x}$.
2. **Hidden Layer:** This is a layer of neurons in the middle of the network.
 - Instead of feeding the raw inputs directly to the final prediction, we can first have a hidden layer compute *new, intermediate features*.
 - For example, this layer might have 3 neurons, each learning to compute a new concept:
 - Neuron 1 $\rightarrow$ (Affordability)
 - Neuron 2 $\rightarrow$ (Awareness)
 - Neuron 3 $\rightarrow$ (Perceived Quality)

- This layer takes the 4 input features ($\vec{x}$) and outputs a new vector of 3 activation values ($\vec{a}$).
- **Why is it "hidden"?** Because the training data provides the inputs (x) and the final output (y), but it does *not* provide the "correct" values for these intermediate features. Their values are "hidden" from the dataset.

3. **Output Layer:** This is the final layer that makes the prediction.

- It takes the activations from the hidden layer (the 3 new features) as its input.
- It performs a final logistic regression computation to output the probability of $y = 1$.

Standard Implementation: Fully Connected Layers

Manually deciding which inputs (e.g., *price*) go to which hidden neurons (e.g., *affordability*) is tedious.

- **Standard Practice:** In a neural network, layers are typically **fully connected**. This means **every neuron** in one layer receives its input from **every activation** in the previous layer.
- The network then *learns* the correct parameters (weights) to determine which inputs are important and which should be ignored for each neuron.

The Power of Neural Networks: Automatic Feature Engineering 🚀

- A neural network can be viewed as a **logistic regression model that automatically learns its own features**.
- In Course 1 (Polynomial Regression), we had to *manually* create new features (like x^2 , x^3 , or $x_1 \times x_2$).
- The **hidden layer** of a neural network performs this feature engineering automatically. It learns the most useful combination or transformation of the input features to make the final prediction easier and more accurate.

Network Architecture

- You are not limited to a single hidden layer. A network can have multiple hidden layers, where the output of one hidden layer becomes the input to the next.
- **Architecture:** The set of choices you make—such as the number of hidden layers and the number of neurons in each layer—is called the network's **architecture**.
- A network with multiple layers is sometimes called a **Multilayer Perceptron (MLP)**.

Neural Networks for Computer Vision

Neural networks are particularly powerful for computer vision tasks, such as face recognition, because of their ability to learn a hierarchy of features directly from pixel data.

Problem Setup: Face Recognition 📷

- **Input ($\vec{x}$):** An image. A computer sees an image as a grid of pixel brightness values. For example, a 1000x1000 pixel image becomes a 1,000,000-element feature vector ($1000 * 1000 = 1,000,000$).
- **Output ($\hat{y}$):** The identity of the person in the picture.

Hierarchical Feature Learning

A deep neural network (one with multiple hidden layers) learns to detect features in a hierarchical way. Earlier layers learn simple features, and later layers combine them into progressively more complex ones.

- **First Hidden Layer:** Learns to detect simple, small-scale features. It's like giving each neuron a small window of the image.
 - *Example:* One neuron might learn to find small vertical edges, another might find horizontal edges, and another might find edges at a 45-degree angle.
- **Second Hidden Layer:** It takes the simple edge features from the first layer as its input and learns to combine them into more complex *parts*.
 - *Example:* By combining various lines and edges, neurons in this layer might learn to detect an **eye**, a **nose**, or the curve of an **ear**.
- **Third Hidden Layer:** It takes the parts from the second layer as input and learns to combine them into even larger shapes.
 - *Example:* Neurons here might learn to detect different **face shapes** (e.g., a round face, an oval face).
- **Output Layer:** It takes the high-level features from the final hidden layer (like "oval face," "has a nose," "has two eyes") and uses them to make the final prediction of the person's identity.

Automatic Feature Learning ✅

The most remarkable property of a neural network is that **it learns this entire feature hierarchy automatically**.

- We do *not* need to tell the network to look for edges, eyes, or noses. We only provide it with the raw pixel data (input) and the correct labels (output).
- During training, the network figures out on its own that detecting edges is useful for

detecting eyes, and detecting eyes is useful for identifying a face.

- This same algorithm, if trained on a different dataset (like pictures of cars), would automatically learn a different hierarchy (e.g., edges → wheels, windows → full car shapes).